Gustavo Carrillo Castro
gustav5@pdx.edu
CS523: Programming Rust

Course Project Proposal: “Rusterm-Audio-VL”, a rust tui audio visualizer

Topic Area: Audio Playback Visualization

Sort of in the realm of emergent behavior, depends on a given implementation. A typical bar chart style one wouldn’t quite fit, but if I spawned particles based off the incoming audio I think it could, it would fall under the cellular automata bucket of things.

Project Vision:
The user can enter a pathway to an audio file and visualize audio based on the contents or connect to the user's mic input. Upon a successful connection to an audio stream we can extract waveform/block samples tied to the audio's bass, mids, highs, treble. This information would be used to set up different audio visualizations. The visualization would be terminal based and use a grid of sorts to spawn different blocks or characters, preferably with a

At a baseline I want your typical bar chart style option. See the attached picture for what I imagine, just rendered in the terminal. Frequency data can be divided into blocks, and each blocks value can just make the column taller or shorter.



Another option I’d like to implement is a rainfall/tetris-y sort of one. Given an n x m grid, we will randomly spawn blocks at the top of the screen of a certain color, the color being based on the intensity of that frequency, with the number of blocks also correlating. Once they hit the bottom of the grid I would like them to persist for a second, allowing them to stack before disappearing. Maybe persistence is timed or it could be like tetris where once a line is entirely full it disappears all at once.

There are other variations I have in mind, like the typical pulsing circles and the like. But I imagine even what I have set out for already could take me longer than expected so I'll leave it at that for now.

Concerns:
There is some domain knowledge to acquire first before getting to the visualization step of things, I need to be able to play audio files/listen to mic input while getting spectrum/waveform information that is then used to actually influence what's displayed. There are rust crates for this step that help one get most of the way there(Rodio, cpal, spectrum-analyzer).

Deciding what to use for writing/drawing to the terminal has overhead in learning the ecosystem. Ratatui seems like the community's choice but again, some initial slowdowns in learning its workflow and program structure. I have started messing around with it already but there are tons of ways to structure information handoff between the app state and draw calls. It looks like the common structure of things is to always draw in the main thread and have background threads that return data to influence app state. Or something simpler like crossterm that is more akin to just printing to the terminal but with color options.

Stretch Goals:
Originally I wanted to build a full terminal mp3 style player, where I could read a directory for music and the ui supported selecting different songs at runtime + audio visualization. But we're halfway through the term as is and that seems unlikely at this moment. That's where something like Ratatui is interesting to me as it has windows and the visualizer could be just one window. Another window could be reserved for a play/stop button, a timeline, skip and previous buttons.

To break things down into smaller steps that could be implemented one at a time.

- Runtime play/stop key options for audio playback.
- Runtime fast forward and reverse, 5 second increments.
- A progress bar tied to what percent of the track has played.
- Window to show different audio files, and let the user pick which to play
- Modify previous step to pull metadata rather than using filenames
- Get music/album covers to display in the terminal.